

SEQUENCE

자료구조 스터디 Week5

학생회

 Made with Gamma

INDEX

01

THEORY

02

IMPLEMENTATION

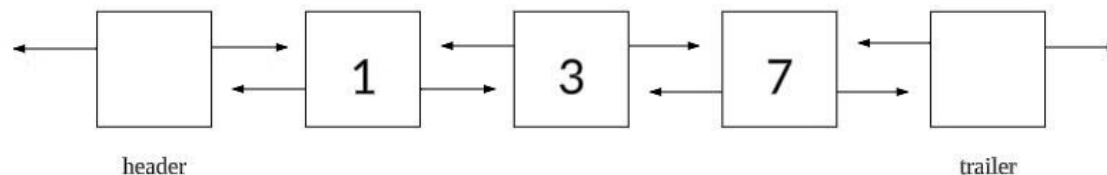
03

PROBLEM SOLVING

시퀀스

시퀀스(Sequence)는 데이터를 **순차적**으로 저장하는 자료구조입니다. 시퀀스는 요소들을 순서대로 저장하며, 각 요소는 특정 위치에 있기에 이 위치를 통해 요소에 접근할 수 있습니다.

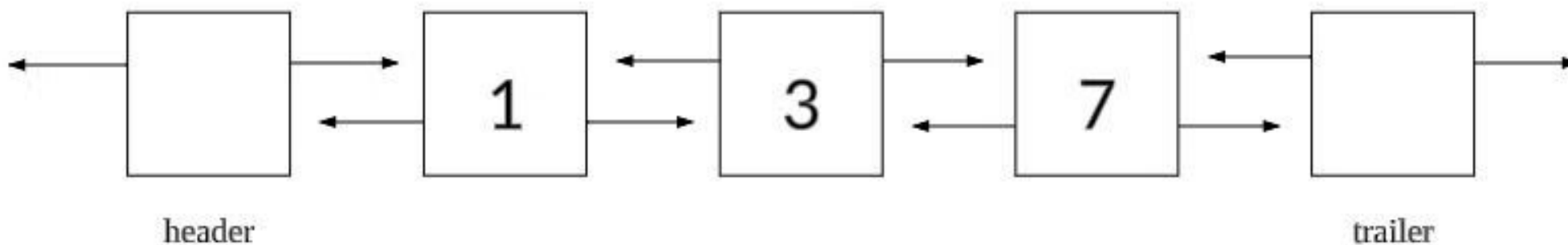
시퀀스의 대표적인 예로는 벡터(Vector), 덱(deque), **리스트(list)**가 있습니다. 저희는 **리스트(list)**에 대해 알아보겠습니다.



리스트의 이해

링크드리스트는 가장 앞 노드를 head, 가장 뒤 노드를 tail로 설정 → 예외처리 복잡

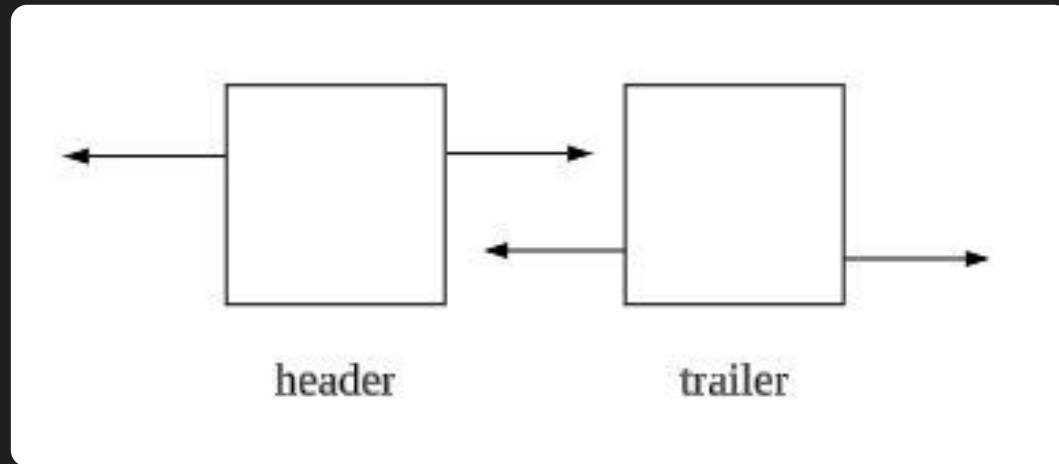
리스트는 header와 trailer를 양 끝 벽으로 설정 → 예외처리 간단



리스트의 이해 - 초기 상태

$\text{header} \rightarrow \text{prev} = \text{nullptr}$ / $\text{header} \rightarrow \text{next} = \text{trailer}$

$\text{trailer} \rightarrow \text{prev} = \text{header}$ / $\text{trailer} \rightarrow \text{next} = \text{nullptr}$

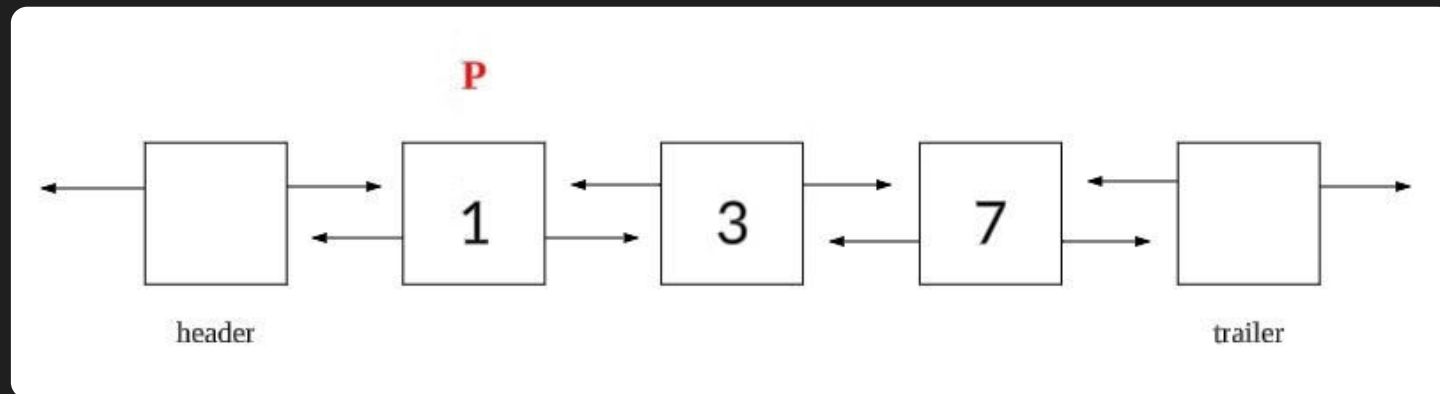


리스트의 이해 - Iterator(반복자)

Iterator(반복자) P는 배열의 Index와 같은 역할 But, P는 노드 그 자체임 ($P \rightarrow \text{value} = 1$)

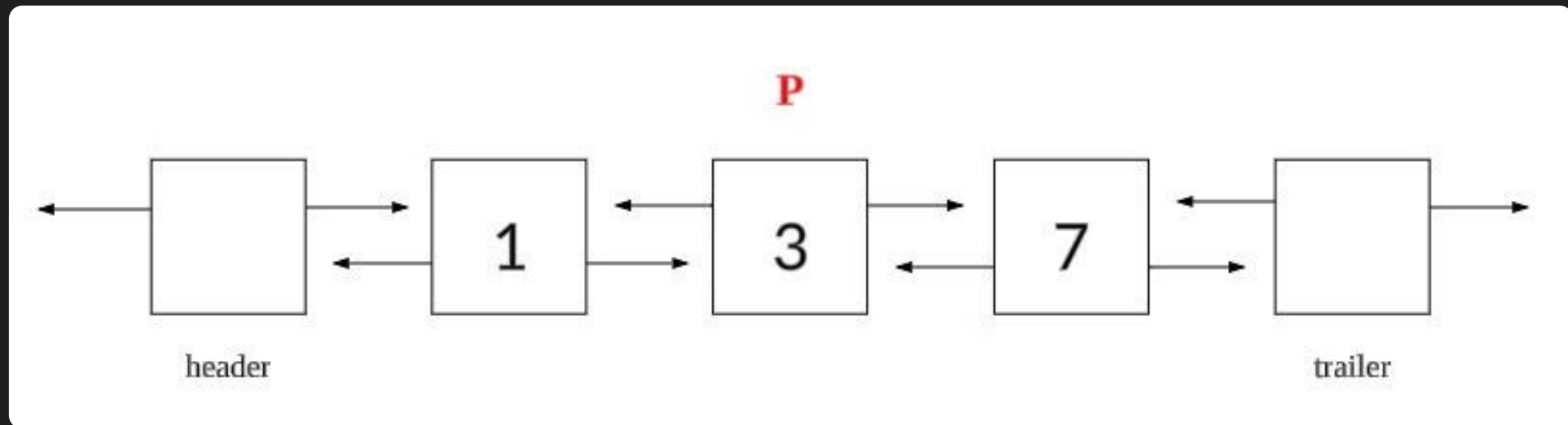
P를 통해 각 요소에 접근 가능, P를 원하는 대로 이동 가능

$P = P \rightarrow \text{next}$ (다음 슬라이드)

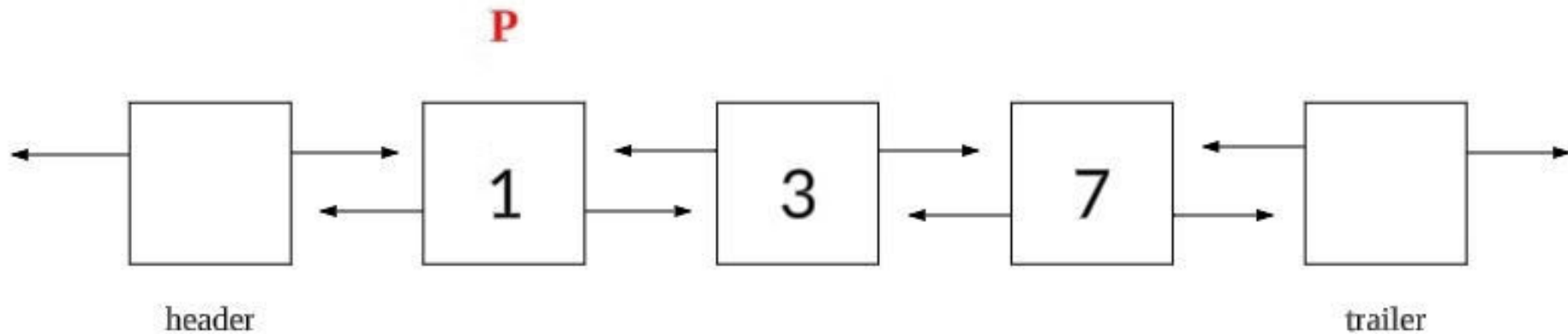


리스트의 이해 - Iterator(반복자)

$P = P \rightarrow \text{prev}$ (다음 슬라이드)



리스트의 이해 - Iterator(반복자)



리스트의 이해 - 결론

링크드리스트

- (시작, 끝 제외) insert, delete 수행 시 curNode로 순회 → 시간복잡도 大(느림)
- 복잡한 예외처리

리스트

- (시작, 끝 제외) insert, delete 수행 시 P를 인자로 전달 → 시간복잡도 小(빠름)
- header, trailer 덕에 간단한 예외처리

리스트는 링크드리스트를 쌘@뽕하게 가공한 것

IMPLEMENTATION



시퀀스의 구현 방법

1

배열

벡터(Vector) - 실습 출제 X

2

링크드 리스트

리스트(List)

리스트의 기본 연산

1 isEmpty

큐가 비어 있는지 확인

2 Size

큐의 크기 조회

3 Begin

header → next 반환

4 End

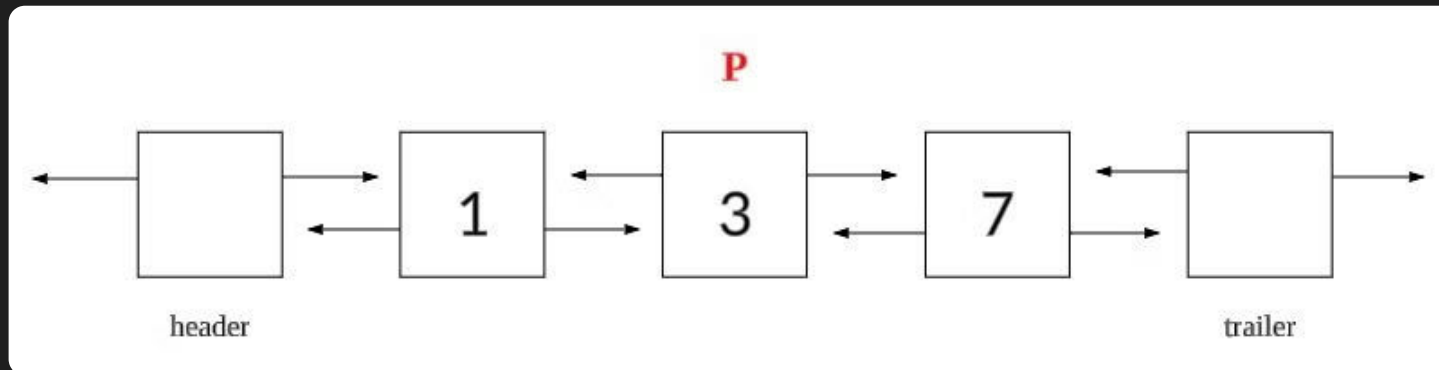
trailer 반환

5 Insert

Iterator P에 요소 추가

6 Erase

Iterator P의 요소 제거



LIST

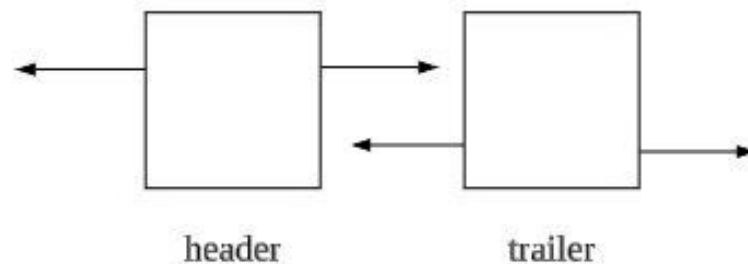
노드 클래스

```
class Node {  
public:  
    int value;  
    Node* prev;  
    Node* next;  
  
    Node() {    // 기본 생성자  
        this->value = 0;  
        prev = next = nullptr;  
    }  
  
    Node(int value) {    // 매개변수 생성자  
        this->value = value;  
        prev = next = nullptr;  
    }  
  
    friend class List;  
};
```

- 값, →, ←
- Iterator 사용을 위해 모두 public 선언
- 기본 생성자와 매개변수 생성자
- friend class → 리스트에서 노드 편하게 사용

멤버 변수

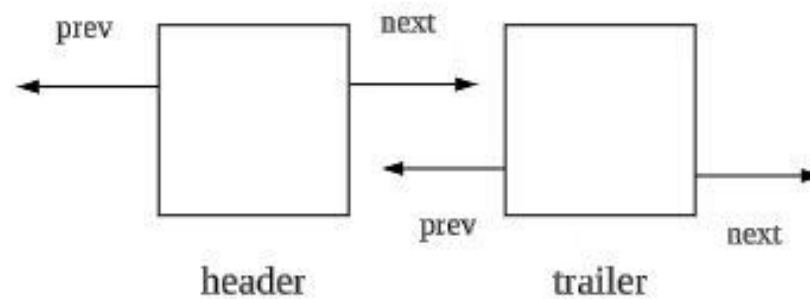
```
class List {  
private:  
    Node* header;    // 가장 앞 벽  
    Node* trailer;   // 가장 뒤 벽  
    int s;           // 리스트의 크기  
};
```



생성자

```
public:
    List() {
        header = new Node();
        trailer = new Node();
        header->next = trailer;
        trailer->prev = header;
        header->prev = trailer->next = nullptr;
        s = 0;
    }
```

초기 상태 - header와 trailer가 앞뒤로 연결되어 있음



isEmpty

```
bool empty() {  
    return (s == 0);  
}
```

- size가 0이라면 True 리턴
- size가 0이 아니라면 False 리턴

size

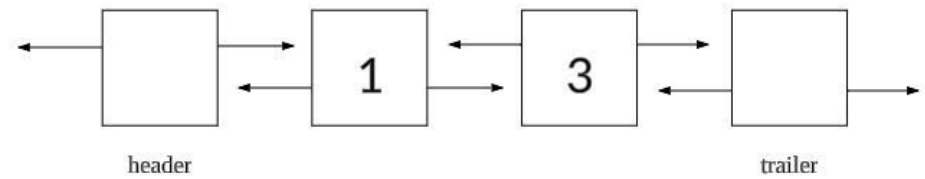
```
int size() {  
    return s;  
}
```

- size 리턴

Begin

```
Node* begin() {  
    return header->next;  
}
```

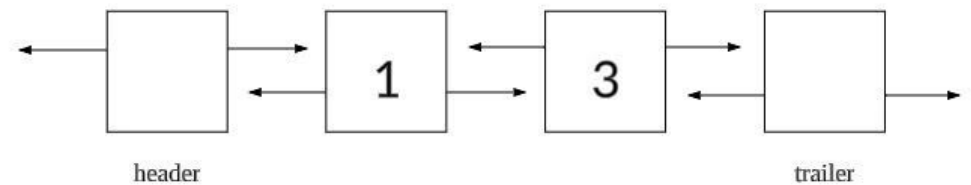
- header→next(노드) 반환



End

```
Node* end() {  
    return trailer;  
}
```

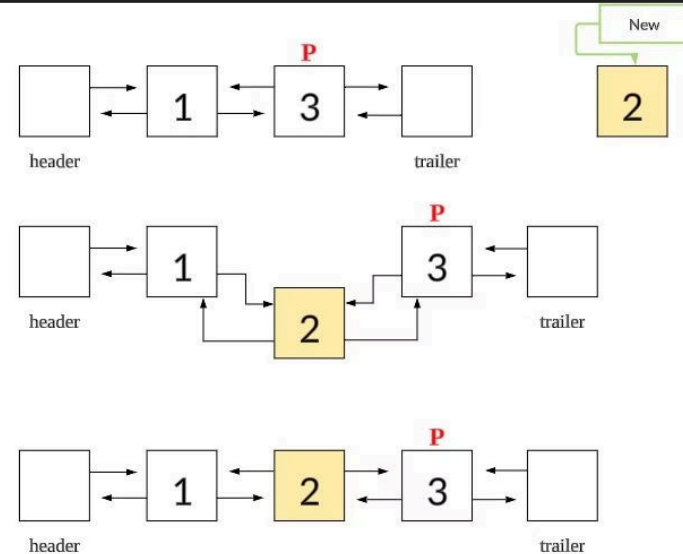
- trailer(노드) 반환



Insert

```
void insert(Node* pos, int value) {  
    Node* new_node = new Node(value);  
  
    new_node->prev = pos->prev;  
    new_node->next = pos;  
  
    pos->prev->next = new_node;  
    pos->prev = new_node;  
  
    s++;  
}
```

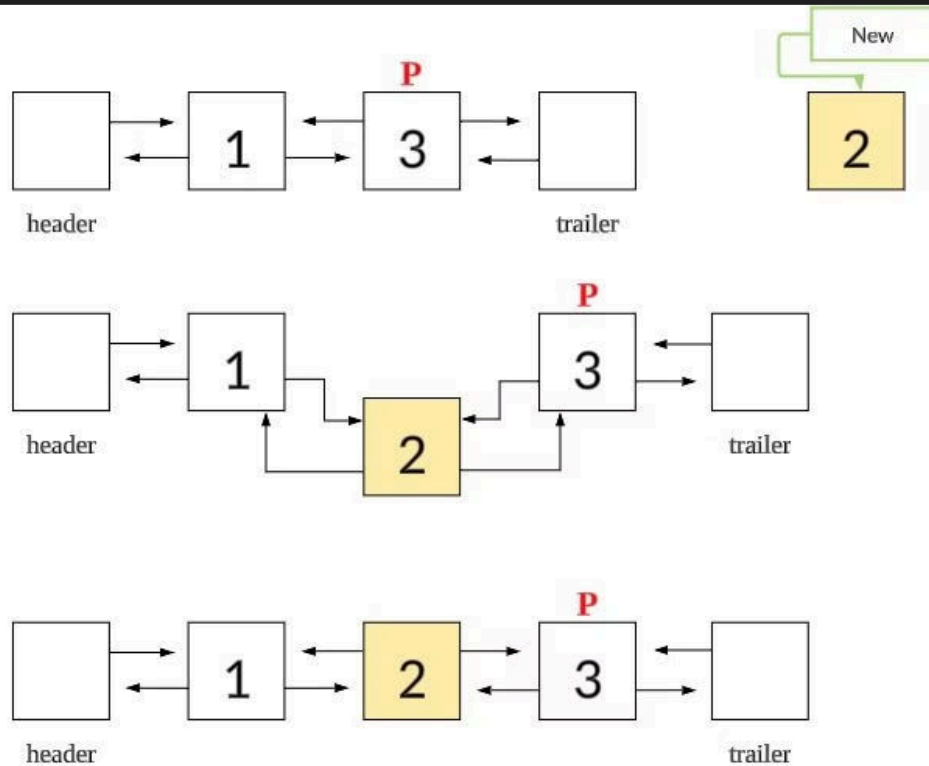
- P 위치에 노드 삽입



- size 1 증가

Insert

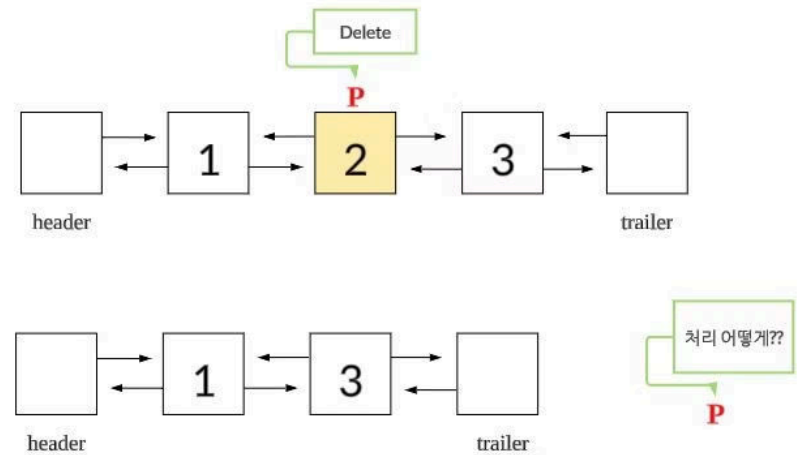
결과적으로 P의 앞에 새로운 노드를 추가한다고 생각해도 무방 ($\because$ P는 노드)



Delete

```
void erase(Node* pos) {  
    pos->prev->next = pos->next;  
    pos->next->prev = pos->prev;  
  
    delete pos;  
    s--;  
}
```

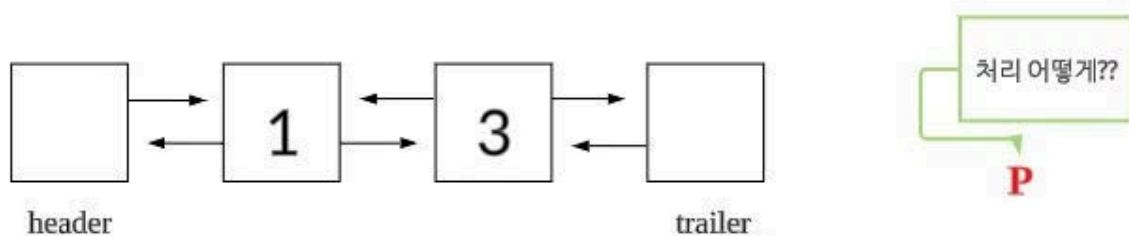
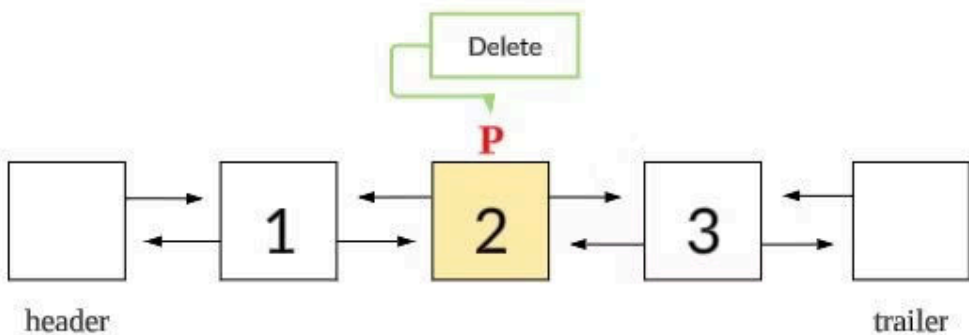
- P 위치의 노드 삭제, P는 문제의 조건에 따라!



- size 1 감소

Delete

노드의 삭제와 함께 사라진 P는 문제의 조건에 따라 재생성 ($\because$ 리스트의 특성 상 P는 꼭 존재해야 함!)



Iterator

Iterator(반복자) P는 Main에 선언

```
int main() {  
    List L;  
    Node* p = L.begin();  
}
```

PROBLEM SOLVING

**THANK'S FOR
WATCHING**